Project 1
Implementation of a Lexical Analyzer
Due on Thursday, October 17, 2024

Note: You need to COMPLETELY follow the instructions in this sheet.

1. **Problem:**

In this project you are requested to use the tool flex to generate a lexical analyzer for a small language Cactus. Cactus contains the following six categories of tokens:

1. This category of tokens includes all identifiers. An identifier is a sequence of underscores, letters, and digits; the first character must be an underscore or a letter. All identifiers are returned as the same token type. Namely, there is only one token type in this category. However, there are an infinite number of possible lexemes for this token type. To distinguish these lexemes, the string of each identifier is returned as the token attribute.
2. This category of tokens includes the following 18 keywords:

   And, Begin, Do, Else, End, EndIf, EndWhile, Exit, If, Set,
   Not, Or, Program, Read, Then, Var, While, Write.

   These keywords are identifiers reserved for special use, and may not be used as general identifiers. Each keyword is returned as a different token type. Namely, there are 18 different token types in this category. Since there is only one lexeme for each token type, no token attribute needs to be returned for each keyword.
3. This category of tokens includes all integer constants. An integer constant consists of a sequence of digits. All integer constants are returned as the same token type. Namely, there is only one token type in this category. However, there are an infinite number of possible lexemes for this token type. To distinguish these lexemes, the integer value of each integer constant is returned as the token attribute.
4. This category of tokens includes the following 14 operators:

   + - * / % = == <> > >= < <= ( )

   Each operator is returned as a different token type. Namely, there are 14 different token types in this category. Since there is only one lexeme for each token, no token attribute needs to be returned for each operator.
5. This category of tokens includes all white spaces. A white space may be a blank (‘ ’), a tab (‘\t’), a newline (‘\n’), or a carriage return (‘\r’). These white spaces are mainly served to separate tokens. These white spaces are ignored

and are not returned.

6. This category of tokens includes all comments. A comment starts with the characters // and ends with a newline. These comments are ignored and are not returned.

The lexical analyzer needs to print the following error message

"Lexical error: line %d: unknown character %c\n"

when an unknown character is encountered. The error message is written to stderr and should include the line number where the error is detected.

The lexical analyzer is generated automatically by flex as the function yylex(). You should add a main function to read a Cactus program from stdin and repeatedly call the yylex() function to recognize tokens until the end of file is returned. When the lexical analyzer is executed with the option "-s", it should print the recognized tokens to stdout. The format for printing tokens is as follows:

Identifiers: print "Identifier:", a space, then the string of the identifier.
Keywords: print "Keyword:", a space, then the string of the keyword.
Integer constants: print "Integer Constant:", a space, then the value of the integer constant.
Operators: print "Operator:", a space, then the string of the operator.

Each token is printed on a separate line. The executable lexical analyzer should be named scanner.

If the Cactus program test1 from stdin is given as follows:

```
// A program to sum 1 to n
Program
Begin
    Var   n
    Var   s

    Read n
    If n < 0 Then
        Write -1
        Exit
    Else
```

```
            Set s = 0
        EndIf
        While n > 0 Do
            Set s = s + n
            Set n = n – 1
        EndWhile
        Write s
    End
```

the output to stdout using –s option should be as follows:

```
Keyword: Program
Keyword: Begin
Keyword: Var
Identifier: n
Keyword: Var
Identifier: s
Keyword: Read
Identifier: n
Keyword: If
Identifier: n
Operator: <
Integer Constant: 0
Keyword: Then
Keyword: Write
Operator: -
Integer Constant: 1
Keyword: Exit
Keyword: Else
Keyword: Set
Identifier: s
Operator: =
Integer Constant: 0
Keyword: EndIf
Keyword: While
Identifier: n
Operator: >
Integer Constant: 0
```

Keyword: Do
Keyword: Set
Identifier: s
Operator: =
Identifier: s
Operator: +
Identifier: n
Keyword: Set
Identifier: n
Operator: =
Identifier: n
Operator: -
Integer Constant: 1
Keyword: EndWhile
Keyword: Write
Identifier: s
Keyword: End

If the Cactus program test2 from stdin is given as follows:

```
// A program to demonstrate input and output
    Program
    Begin
        Read #n
        Write @s
    End
```

the output to stdout using –s option should be as follows:

Keyword: Program
Keyword: Begin
Keyword: Read
Identifier: n
Keyword: Write
Identifier: s
Keyword: End

the output to stderr should be as follows:

Lexical error: line 4: unknown character #
Lexical error: line 5: unknown character @

2. Tool Installation:

Tools flex and bisen can be installed on Windows or Ubuntu. Guides for tool installation have been posted in Chapter 2 on eCourse2 site.

3. Handing in your program:

The executable file should be named "scanner." In Windows,

```
flex cactus.l
gcc -o scanner lex.yy.c
type test1.txt | .\scanner.exe
```

In Ubuntu,

```
flex cactus.l
gcc -o scanner lex.yy.c
scanner < test1
```

To turn in this project, upload a flex file cactus.l to eCourse2 site.

3. Grading

The grading is based on the correctness of your program. The correctness will be tested by a number of test cases designed by the instructor and teaching assistants.

It is best to incrementally build your program so that you always have a partially-correct working program. It is also best to construct a shell script to systematically test your program.